6. Hafta

FastAPI I: HTTP'yi Koda Dökmek

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- Python sanal ortamı (venv) kurup bağımlılıkları `pip` ile kurarak, `uvicorn` ile çalıştırılan ilk FastAPI uygulamanızı yazabilecek; yol, sorgu ve gövde parametrelerini tip ipuçlarıyla tanımlayıp FastAPI'nin bunları otomatik dönüştürüp doğruladığını açıklayabileceksiniz;
- `BaseModel` tabanlı Pydantic modelleri, `Field` kısıtları ve giriş/çıkış modeli ayrımıyla (`EtkinlikOlustur` vs `Etkinlik`) tanımlayabilecek; camelCase/snake_case köprüsünü (`alias_generator=to_camel`) kurup 422 doğrulama hatasının anatomisini (`loc`/`msg`/`type`) yorumlayabileceksiniz;
- Bellekte bir `dict` deposu üzerinde CRUD uç noktaları (`POST`→201, `GET`→200, `PUT`/`PATCH`→200, `DELETE`→204) yazabilecek, `HTTPException(404)` ile eksik kayıt hatası, `409 Conflict` ile koltuk rezervasyonu iş kuralını uygulayabilecek ve rotaları `APIRouter` ile varlık başına ayırabileceksiniz;
- `/docs` ve `/openapi.json`'ı okuyup Hf.3'ün TS tipleriyle karşılaştırabilecek; `Depends` ile bir depo nesnesini rota fonksiyonlarına enjekte edebilecek ve `TestClient` ile ilk API testlerinizi (`pytest`) yazabileceksiniz;
- Bu becerileri birleştirip Etkinlik/Koltuk/Bilet için bellekte çalışan, koltuk rezervasyonu iş kuralını uygulayan, kendi kendini belgeleyen (`/docs`) ve testlerle doğrulanmış bir CRUD API'si oluşturarak Etkinlik/Bilet Sistemine ilk sunucu katmanını ekleyebileceksiniz.

Geçen hafta `App`'in `SepetContext`'inde tuttuğunuz `Bilet[]` nesneleri yalnızca tarayıcı sekmenizin belleğinde yaşıyordu — sayfayı yenilediğinizde bu biletler kayboluyordu, bir arkadaşınız aynı siteyi başka bir sekmede açtığında bu biletleri hiç görmüyordu, çünkü veri hiçbir zaman kendi bilgisayarınızın dışına çıkmadı. Bu hafta bu sınırı ortadan kaldıran bir adım atıyorsunuz: ilk kez gerçek bir sunucu yazıyorsunuz. Hf.1'in yalnızca kavram düzeyinde gördüğünüz istemci-sunucu modeli, HTTP metotları ve durum kodları, bu hafta kendi yazdığınız Python kodunda somutlaşacak; Hf.3'ün `Etkinlik`/`Koltuk`/`Bilet` arayüzleri bu haftanın Pydantic modellerinde birebir karşılığını bulacak. Dört ünite boyunca sırasıyla ilk FastAPI uygulamanızı nasıl çalıştıracağınızı ve yol/sorgu/gövde parametrelerini nasıl tanımlayacağınızı (Ünite 1), gelen veriyi çalışma zamanında doğrulayan Pydantic

modellerini (Ünite 2), bellekte bir depo üzerinde tam bir CRUD API'sini — koltuk rezervasyonu iş kuralı dahil — nasıl yazacağınızı (Ünite 3) ve son olarak bu API'nin kendi kendini nasıl belgelediğini, bir depo nesnesinin rota fonksiyonlarına nasıl enjekte edildiğini ve ilk testlerinizi nasıl yazacağınızı (Ünite 4) göreceksiniz.

Bu haftanın bilinçli sınırları vardır. Veri hâlâ bir veritabanında değil, yalnızca sunucu sürecinin belleğinde tutulur — sunucu yeniden başladığında (`--reload` dahil) tüm veri kaybolur; kalıcılık Hf.8'in konusudur. Ön yüzünüz (React) bu haftaki API'ye hiç bağlanmaz — Hf.5'in `App`'i hâlâ Hf.3'ün sabit (mock) dizisini kullanmaya devam eder; bağlantı, `fetch` ve CORS ile birlikte Hf.7'nin konusudur. Kimlik doğrulama/roller de bu hafta yoktur (Hf.10); `kullaniciAdi` alanı şimdilik düz metindir. Bu sınırların hiçbiri bir eksiklik değildir — her biri, bu haftanın tek işine (HTTP'yi Python koduna dökmek) odaklanmak için bilinçli olarak bırakılmıştır.

6.1 İlk Uygulama, Yol/Sorgu/Gövde Parametreleri

1 • Ön-öğretim — Neden Önemli?

Hf.5'te oluşturduğunuz `Bilet[]` nesneleri yalnızca tarayıcı sekmenizin belleğinde yaşıyordu — sayfayı yenilediğinizde ya da bir arkadaşınız aynı siteyi başka bir sekmede açtığında bu biletleri göremezdi, çünkü veri hiçbir zaman bilgisayarınızın dışına çıkmadı. Bu hafta ilk kez gerçek bir sunucu yazıyorsunuz: aynı veriye birden fazla istemci (iki farklı terminal penceresi, `curl`, ileride Hf.7'nin React uygulaması) erişebilecek — "aynı etkinliği iki kullanıcı aynı anda görür mü?" sorusunun cevabı artık EVETtir, çünkü veri artık paylaşılan, ortak bir sunucu sürecinde yaşayacak.

1 • Ön-öğretim — Ön Bilgi

Hf.1 Ünite 2-4'ün istemci-sunucu modelini, HTTP metot/yol/durum kodu anatomisini ve `curl` deneyimini hatırlayın — bu hafta aynı `curl` komutlarını, ama artık kendi sunucunuza göndereceksiniz. YG II'nin fonksiyon imzalarına tip belirtme alışkanlığını da hatırlayın (C++'ın statik tipleriyle tanıdık bir şey) — Python'da tip ipuçları bu haftanın FastAPI kodunun temelini oluşturuyor. Son olarak Hf.3 Ünite 3'ün `Etkinlik`/`Koltuk`/`Bilet` arayüzlerini hatırlayın — bu haftanın Pydantic modelleri bunların birebir karşılığı olacak (ayrıntı Ünite 2'de).

Bu ünitede önce bir FastAPI projesini nasıl ayağa kaldıracağınızı, sonra da ilk uç noktanızı — ve bu uç noktanın URL'den, sorgu dizesinden ve istek gövdesinden nasıl veri aldığını — göreceksiniz.

6.1.1 Sanal ortam (venv) ve bağımlılıklar

Bu dersten önce Python'ı zaten tanıyorsunuz, ama muhtemelen tek bir küresel (*global*) Python kurulumuyla çalıştınız. Birden fazla proje aynı bilgisayarda geliştiğinde bir sorun ortaya çıkar: bir proje `fastapi` paketinin 0.141 sürümünü isterken, başka bir proje aynı paketin daha eski bir sürümünü isteyebilir. İkisini de küresel olarak kurmaya çalışmak, ikinci kurulumun birincisinin yerini almasına — yani ilk projenin bozulmasına — yol açar. Bu sorunu çözmek için Python bir **sanal ortam (venv)**

sanal ortam (venv): Bir projeye özel, diğer projelerden izole Python paket kurulumu (*virtual environment*).

mekanizması sunar: her proje kendi izole paket kümesini, projenin kendi klasöründe tutar.

Analoji

Sanal ortamı, bir mutfakta birden fazla tarif (proje) hazırlarken her tarifin kendi malzeme setini (paket sürümlerini) ayrı bir çekmecede tutmaya benzetin: bir tarifin unu diğerininkiyle karışmaz, her çekmece kendi tarifine göre doldurulur ve boşaltılır.

Bir sanal ortam şu komutla oluşturulur (proje klasörünün içinde çalıştırılır):

```
Terminal
$ python -m venv .venv
```

Bu komut, proje klasörünüzün içinde `.venv` adlı bir alt klasör oluşturur — bu klasör kendi izole Python yorumlayıcısını ve paket kurulumlarını barındırır. Sanal ortamı kullanmaya başlamak için önce aktive etmeniz gerekir; işletim sistemine göre komut değişir:

```
Terminal
$ source .venv/bin/activate        # macOS / Linux
$ .venv\Scripts\Activate.ps1       # Windows PowerShell
```

Aktivasyondan sonra terminal isteminizin (*prompt*) başında `(.venv)` gibi bir işaret belirir — bu, o an çalıştırdığınız `python`/`pip` komutlarının küresel kuruluma değil, bu projenin izole ortamına gittiğini gösterir. Bağımlılıkları kurmak için:

```
Terminal
$ pip install -r requirements.txt -r requirements-dev.txt
```

Burada iki ayrı dosya dikkatinizi çekmiş olmalı. Bunun nedeni, bir sunucunun çalışma zamanında gerçekten ihtiyaç duyduğu paketlerle (`fastapi`, `uvicorn`, `pydantic`), yalnızca geliştirme/test sırasında gereken paketlerin (`pytest`, `httpx`) ayrı tutulmasıdır — üretime dağıtılan bir sunucunun test araçlarını kurması gerekmez, bu da dağıtım paketini gereksiz yere büyütmez:

```
1 fastapi~=0.141
2 uvicorn[standard]~=0.52
3 pydantic~=2.13
```

```
1 -r requirements.txt
2 pytest~=9.1
3 httpx~=0.28
```

`requirements-dev.txt`'nin ilk satırındaki `-r requirements.txt` dikkatinizi çeksin — bu satır, geliştirme dosyasının çalışma zamanı dosyasını içine aldığını (*include*) belirtir; yani `requirements-dev.txt`'i kurduğunuzda her iki dosyanın paketleri de gelir, `requirements.txt`'i ayrıca belirtmeniz gerekmez.

> **İpucu**
>
> Sürüm numaralarındaki `~=` işareti (ör. `fastapi~=0.141`) "uyumlu sürüm" operatörüdür: `0.141` ana/minör dalında kalıp yama sürümlerini (`0.141.1`, `0.141.2`...) kabul eder, ama `0.142`'ye atlamaz — npm'in `^` işaretine kavramsal karşılıktır.

6.1.2 FastAPI ile `uvicorn`'un ilişkisi

ASGI sunucusu: Python'da asenkron web uygulamalarını çalıştıran, TCP/HTTP protokolünü konuşan süreç (*ASGI server*).

FastAPI bir çerçevedir (*framework*) — hangi URL yoluna hangi Python fonksiyonunun cevap vereceğini tanımlamanıza yarar. Ama FastAPI tek başına bir TCP bağlantısını dinleyemez, HTTP protokolünü konuşamaz — bu işi **ASGI sunucusu** olan `uvicorn` üstlenir. Hf.1 Ünite 2'de "web sunucusu" kavramını soyut olarak gördünüz; `uvicorn` bu kavramın somut, kendi bilgisayarınızda çalışan bir örneğidir.

> **Analoji**
>
> FastAPI ile `uvicorn` ilişkisini bir gişe binasıyla gişe memurunun kural kitapçığına benzetin: bina (`uvicorn`) kapıyı açar, gelen ziyaretçiyi (isteği) içeri alır; ama hangi ziyaretçinin hangi pencereye yönlendirileceğini, pencerede ne sorulacağını belirleyen kural kitapçığı FastAPI'nin yazdığınız kodudur.

Sunucuyu şu komutla çalıştırırsınız:

```
Terminal
$ uvicorn app.main:app --reload
```

Bu komutun her parçasının bir anlamı vardır: `app.main` Python modül YOLUdur (`app/main.py` dosyasına karşılık gelir); `:app` bu modülün içinde tanımlı FastAPI nesnesinin ADIdır; `--reload` ise kaynak dosyalarınız değiştiğinde sunucunun otomatik olarak yeniden başlamasını sağlar — yalnızca geliştirme sırasında kullanılır, üretimde kapatılır (kaynak dosyalar üretimde değişmediği için gereksizdir, ayrıca performans maliyeti getirir).

6.1.3 İlk uç nokta ve yol parametresi

Uygulama nesnesi ve ilk uç noktanız `app/main.py` dosyasında şöyle tanımlanır:

```python
app = FastAPI(
    title="KTÜ Etkinlik ve Bilet Sistemi API",
    version="0.1.0",
)
```

```python
@app.get("/")
def kok_rota() -> dict[str, str]:
    return {"mesaj": "KTÜ Etkinlik ve Bilet Sistemi API'sine hoş geldiniz"}
```

`@app.get("/")` bir dekoratördür — altındaki `kok_rota` fonksiyonunun, `GET` metoduyla `/` yoluna gelen isteklere cevap vereceğini FastAPI'ye bildirir. Sunucu çalışırken bu uca gerçekten istek gönderdiğinizde:

Terminal

```
$ curl -i http://127.0.0.1:8000/
```

Beklenen Çıktı

```
HTTP/1.1 200 OK
content-type: application/json

{"mesaj":"KTÜ Etkinlik ve Bilet Sistemi API'sine hoş geldiniz"}
```

Hf.1 Ünite 4'te `curl -v https://www.example.com` uzak bir sunucuya gitmişti; bu satırdaki `curl` komutu ise yerel makinenizde sizin yazdığınız bir Python fonksiyonunun ürettiği gerçek bir cevaba gider — istek/cevap anatomisi artık soyut değil, kendi kodunuzun somut çıktısıdır.

Şimdi URL'nin değişken bir parçasını yakalayan bir rotaya bakın:

```python
@app.get("/etkinlikler/{etkinlik_id}")
def etkinlik_detay(etkinlik_id: int):
    ...
```

yol parametresi: URL yolunun {} içinde tanımlı, değişken bir parçasını yakalayan parametre (*path parameter*).

Yol şablonundaki `{etkinlik_id}` bir **yol parametresi**dir — `/etkinlikler/1`, `/etkinlikler/42` gibi farklı yollar aynı fonksiyona eşlenir, yalnız `etkinlik_id` her seferinde farklı bir değer alır. Fonksiyon parametresine verilen `: int` tip ipucu kritiktir: FastAPI URL'deki metni (her zaman bir dizedir) otomatik olarak `int`'e çevirmeye çalışır — başarılı olursa fonksiyonunuz zaten bir tamsayı ile çağrılır, elle `int(...)` dönüşümü yapmanız gerekmez.

Analoji

Yol parametresini bir sıra numarasındaki değişken kısma benzetin: "138. gişe" tabelasındaki sayı her seferinde değişir ama tabelanın biçimi (gişe adı + numara) sabittir — `{etkinlik_id}` da böyle, her istekte farklı bir değer alır ama yolun geri kalanı sabit kalır.

6.1.4 Sorgu parametresi

Yol şablonunda geçmeyen, ama fonksiyon imzasına eklediğiniz bir parametre farklı bir role bürünür:

```python
@app.get("/etkinlikler")
def etkinlik_liste(mekan: str | None = None):
    ...
```

sorgu parametresi: URL'nin ?ad =deger biçimindeki, isteğe bağlı ek parametresi (*query parameter*).

`mekan` yol şablonunun (`/etkinlikler`) hiçbir yerinde geçmediği için FastAPI bunu otomatik olarak bir **sorgu parametresi** sayar — yani `?mekan=...` biçiminde URL'nin sonuna eklenen, isteğe bağlı bir değer olarak ele alınır. Kural basittir: yol şablonunda `{}` içinde adı geçen her parametre yol parametresidir; geçmeyen her parametre sorgu parametresidir. Varsayılan değeri `None` olduğu için `mekan` belirtilmeden (`GET /etkinlikler`) de, belirtilerek (`GET /etkinlikler?mekan=Salon`) de çağrılabilir.

Analoji

Sorgu parametresini bir form dilekçesindeki isteğe bağlı kutucuğa benzetin: dilekçenin zorunlu alanları (yol) doldurulmadan form kabul edilmez, ama "ek not" gibi bazı kutucuklar (sorgu parametreleri) boş bırakılabilir — boş bırakıldığında varsayılan bir değerle işleme devam edilir.

6.1.5 Gövde parametresi: kısa bir önizleme

`POST`/`PUT` gibi isteklerin genellikle bir JSON gövdesi vardır — yeni bir etkinliğin başlığı, tarihi, kapasitesi gibi birden fazla alanı taşıyan bir veri paketi. Bu veriyi almak için fonksiyon parametresinin tipinin bir Pydantic `BaseModel` alt sınıfı olması gerekir; buna **gövde parametresi** denir. Bu ünite bu kavramı yalnızca tanıtır — bir modelin tam olarak nasıl tanımlandığını, hangi alanların hangi kısıtları taşıyacağını Ünite 2'de ayrıntılı göreceksiniz; burada kod örneği içermez.

gövde parametresi: İstek gövdesinden (genelde JSON) okunan, bir modele bağlanan parametre (*request body parameter*).

Ortak Hata / Tuzak

Yeni başlayan bir FastAPI geliştiricisi genellikle `"abc"` gönderildiğinde `404` bekler — "kayıt bulunamadı" sezgisiyle. Ama bir önceki örnekte gördüğünüz gibi, tip dönüşümü başarısız olduğunda FastAPI `422 Unprocessable Entity` döner, `404` değil. `404` yalnızca yolun kendisi hiç tanımlı değilse (istediğiniz rota hiç yazılmamışsa) ortaya çıkar; burada yol tanımlıdır, yalnızca parametrenin tipi uymamaktadır. Bu ayrımı net tutmak, ilerleyen ünitelerde kendi `404` hatalarınızı (Ünite 3) doğru yerde kullanmanız için önemlidir.

Bu farkı gerçek bir istekle doğrulayın:

Terminal

```
$ curl -i http://127.0.0.1:8000/etkinlikler/abc
```

Beklenen Çıktı

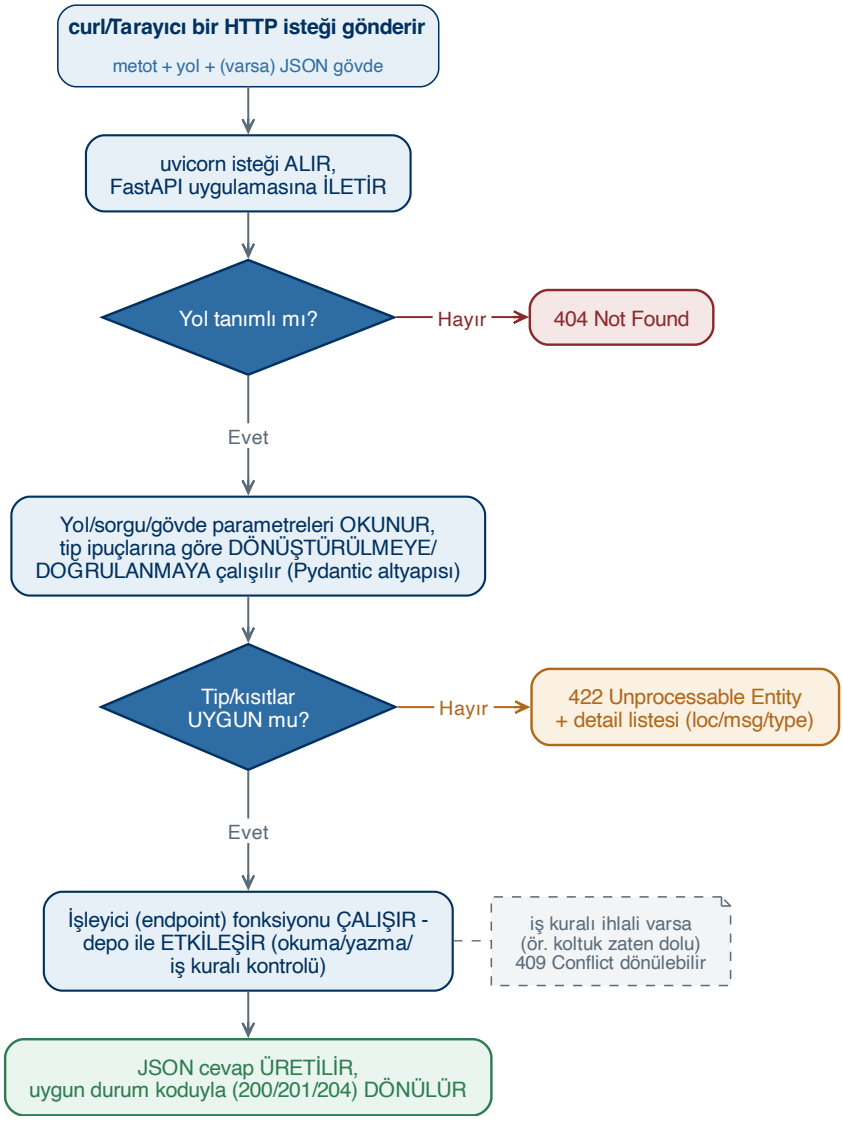
```
HTTP/1.1 422 Unprocessable Entity
content-type: application/json

{"detail":[{"type":"int_parsing","loc":["path","etkinlik_id"],
"msg":"Input should be a valid integer, unable to parse string as an
integer","input":"abc"}]}
```

Şekil 6.1, bir isteğin `uvicorn`'a ulaşmasından, yol eşlemesinden, parametre dönüşümünden/doğrulamasından geçip ya bir JSON cevaba ya da bir hata dalına (`404`/`422`) ulaşmasına kadar bu sürecin tamamını gösterir — Ünite 2'de bu akışın gövde/model doğrulama kısmına daha yakından bakacaksınız.

3 • Katılım — Şimdi Siz Deneyin

Terminalde `uvicorn app.main:app --reload` ile sunucuyu başlatın; `curl -i http://127.0.0.1:8000/` ile kök uç noktaya istek gönderin. Ardından `curl -i http://127.0.0.1:8000/etkinlikler/abc` ile geçersiz bir yol parametresi deneyin, durum kodunu gözlemleyin.



Şekil 6.1: Bir HTTP isteğinin `uvicorn`'dan işleyici fonksiyona kadar geçtiği yol eşleme ve doğrulama adımları; başarı ile `404`/ `422` hata dalları.

Geribildirim. Kök uç nokta `200 OK` ile hoş geldin JSON'unu döner; `/etkinlikler/abc` isteği `422 Unprocessable Entity` döner çünkü `"abc"` bir `int`'e çevrilemez — FastAPI bu dönüşümü/doğrulamayı işleyici fonksiyonunuzu çağırmadan önce, tamamen otomatik olarak yapar; siz hiçbir `if` yazmadınız.

Kendinizi Sınayın 6.1

`/etkinlikler/{etkinlik_id}` yoluna `etkinlik_id: int` tip ipucuyla tanımlı bir rotaya `/etkinlikler/abc` isteği gönderirseniz ne olur, hangi durum kodu döner — neden `404` değil?

5 • İzleme — Aktarım — Başka Nerede?

Bugün kurduğunuz `venv+pip+uvicorn` iş akışı, dönem boyunca yazacağınız her Python sunucu projesinde (Hf.8, Hf.10, Hf.13...) aynen tekrarlanacak; yol/sorgu parametresi ayrımı, ileride yazacağınız her yeni uç noktada (ör. Hf.10'un `/kullanicilar/{id}` rotası) aynı sözdizimiyle karşınıza çıkacaktır.

6.2 Pydantic Modelleri ve Doğrulama

1 • Ön-öğretim — Neden Önemli?

Ünite 1'de bir yol parametresine yanlış tipte veri gönderdiğinizde FastAPI'nin otomatik bir hata ürettiğini gördünüz. Ama bir `POST` isteğinin gövdesindeki — negatif bir kapasite, boş bir başlık gibi — yanlış veriyi kim, nasıl yakalayacak? Her rota içinde elle `if kapasite <= 0: ...` yazmak, her yeni rota için tekrar tekrar yazılması gereken, unutulmaya açık bir iştir — bu ünite bu tekrarı ortadan kaldıran mekanizmayı gösteriyor.

1 • Ön-öğretim — Ön Bilgi

Hf.3 Ünite 3'ün `interface Etkinlik { id, baslik, tarih, mekan, kapasite, biletFiyati }` tanımını hatırlayın — bu haftanın Pydantic modeli bunun birebir karşılığı olacak. Hf.1 Ünite 4'ün durum kodu ailelerini (4xx istemci hatası) de hatırlayın; bu ünitede özellikle `422` durum kodunu ayrıntılı göreceksiniz.

6.2.1 `BaseModel`: tip ipuçlarından doğrulama mantığına

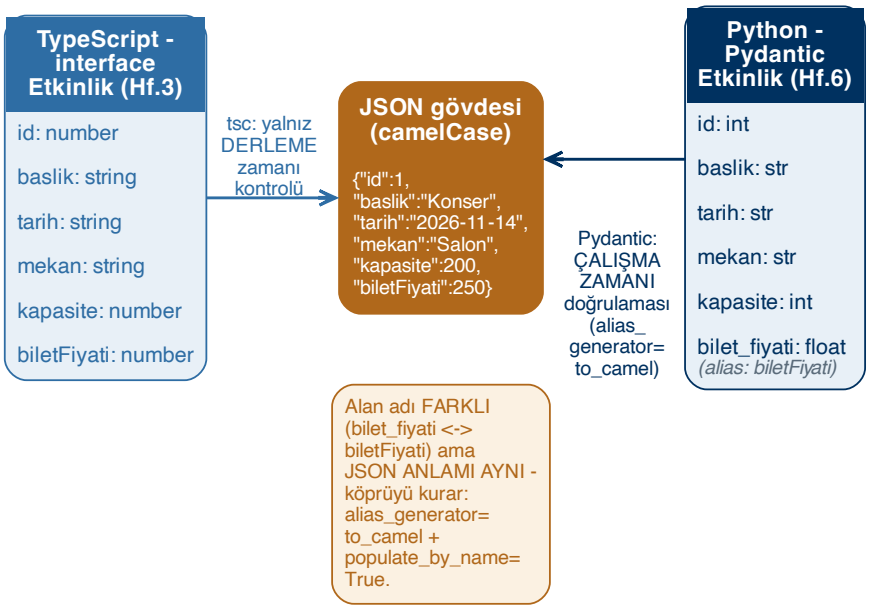
Hf.3'te bir `interface Etkinlik` yazdığınızda, TypeScript bunu yalnızca derleme zamanında kontrol ediyordu — `tsc` derledikten sonra, çalışan JavaScript kodu içinde bu arayüze artık hiçbir şekilde başvurulmuyordu; sunucudan gelen veri, biçimi ne olursa olsun, doğrudan "`Etkinlik`" olarak kabul ediliyordu. Python'da durum farklıdır: Pydantic'in **Pydantic modeli**, bir sınıfın alanlarını tip ipuçlarından türeterek, bu sınıftan bir nesne oluşturulduğu her seferinde — yani çalışma zamanında — gelen veriyi denetler, uymayan veriyi reddeder.

Pydantic modeli: Tip ipuçlarından otomatik doğrulama/ayrıştırma mantığı üreten Python sınıfı (*Pydantic model*).

Analoji

Pydantic doğrulamasını bir gişe memurunun, size verdiğiniz formu imzalamadan önce her alanı kontrol etmesine benzetin: yaş kutusuna harf yazmışsanız, kapasiteyi negatif girmişseniz, memur formu kabul etmez, size hangi alanın yanlış olduğunu söyler — form işleme hiç girmez.

Şekil 6.2, Hf.3'ün TS `interface Etkinlik`'i ile bu haftanın Pydantic `Etkinlik` modelini yan yana gösterir — iki dilde yazılan aynı veri şeklinin, aynı camelCase JSON gövdesinde nasıl buluştuğunu izleyin.



Şekil 6.2: Hf.3'ün TS `interface Etkinlik`'i ile bu haftanın Pydantic `Etkinlik` modelinin camelCase/snake_case köprüsü üzerinden eşlemesi.

Bu köprünün Python tarafındaki taban sınıfı şöyle tanımlanır:

```
class KamelModel(BaseModel):

    model_config = ConfigDict(alias_generator=to_camel, populate_by_name=True)
```

`model_config` satırındaki `ConfigDict(...)` çağrısı, `KamelModel`'den türeyen her sınıf için iki şey yapar. Birincisi `alias_generator=to_camel`: her `snake_case` alan adı için otomatik bir camelCase **takma ad (alias)** üretir (`bilet_fiyati` → `biletFiyati`); FastAPI bir modeli JSON'a çevirirken varsayılan olarak bu takma adı kullanır, bu sayede sunucunun JSON çıktısı Hf.3'ün TS arayüzleriyle birebir eşleşir. İkincisi `populate_by_name=True`: bu ayar olmasaydı, model yalnızca takma adla (`biletFiyati`) doldurulabilirdi; bu ayarla model hem takma adla (JSON'dan gelen veri) hem Python alan adıyla (`bilet_fiyati`, sunucu içi kodda) doldurulabilir.

takma ad (alias): Bir modelin dış dünyaya (JSON) farklı bir adla göründüğü alan eşlemesi (*alias*).

Analoji

camelCase/snake_case köprüsünü, aynı kişinin iki farklı kurumda farklı biçimde yazılan adına benzetin (nüfus kaydında büyük harf, banka sisteminde küçük harf) — biçim farklıdır, anlam aynıdır; bir eşleme tablosu (`alias`) ikisini

birbirine bağlar.

6.2.2 Giriş/çıkış modeli ayrımı ve `Field` kısıtları

`Etkinlik` varlığı için modeller şöyle tanımlanır:

```
class EtkinlikOlustur(KamelModel):

   baslik: str = Field(min_length=1)
   tarih: str
   mekan: str = Field(min_length=1)
   kapasite: int = Field(gt=0)
   bilet_fiyati: float = Field(gt=0)

class Etkinlik(EtkinlikOlustur):

   id: int
```

Giriş modeli: İstemciden gelen, henüz kimliği (id) atanmamış veri şekli (*input model*).

Burada iki ayrı sınıf dikkatinizi çeksin: **Giriş modeli** olan `EtkinlikOlustur`, istemcinin gönderdiği, henüz `id`'si olmayan veriyi temsil eder; `Etkinlik` ise `EtkinlikOlustur`'dan türer ve yalnızca bir `id: int` alanı ekler — bu da **çıkış modeli** olur, sunucunun döndürdüğü, kimliği atanmış tam nesnedir. Bu ayrım rastgele değildir — Hf.1 Ünite 3'ün "sunum/iş/veri katmanı" ayrımının somut bir yansımasıdır: dışarıdan gelen veri şekli ile dışarıya giden veri şekli aynı olmak zorunda değildir.

çıkış modeli: Sunucunun döndürdüğü, kimliği atanmış tam veri şekli (*output model*).

Analoji

Giriş/çıkış modeli ayrımını bir sipariş fişinin doldurma tarafı ile mutfak/kasa çıktısının farklı bilgiler taşımasına benzetin: siz siparişi verirken (giriş) yalnızca ürün ve adet yazarsınız, size verilen fiş (çıkış) ise bir sipariş numarası ve toplam tutar da içerir — aynı siparişin iki yüzü farklı bilgi taşıyabilir.

Alan tanımlarındaki `Field(gt=0)` ve `Field(min_length=1)` çağrılarına dikkat edin — bunlar yalnızca bir alanın tipini değil, değer aralığını da doğrular: `kapasite: int = Field(gt=0)` "`kapasite` bir tamsayı olmalı ve 0'dan büyük olmalı" der. `Field`'ın sık kullanılan diğer kısıtları arasında `ge`/`lt`/ `le` (büyük-eşit/küçük/küçük-eşit) ve `max_length` (bir metnin azami uzunluğu) sayılabilir. `Koltuk` ve `Bilet` için de aynı giriş/çıkış ayrımı, benzer `Field` kısıtlarıyla kurulur:

```
KoltukDurumu = Literal["bos", "secili", "dolu"]

class KoltukOlustur(KamelModel):

   sira: int = Field(gt=0)
   koltuk_no: int = Field(gt=0)

class Koltuk(KoltukOlustur):

   id: int
   etkinlik_id: int
   durum: KoltukDurumu = "bos"

class KoltukGuncelle(KamelModel):

   durum: KoltukDurumu
```

```python
BiletDurumu = Literal["olusturuldu", "iptal"]

class BiletOlustur(KamelModel):

    koltuk_id: int
    kullanici_adi: str = Field(min_length=1)

class Bilet(BiletOlustur):

    id: int
    satin_alma_zamani: str
    durum: BiletDurumu = "olusturuldu"
```

6.2.3 `Literal` ile durum tipleri

`koltuk_modelleri` kesitinin başındaki `KoltukDurumu` tanımı — `Literal["bos", "secili", "dolu"]` — Hf.3'ün `type KoltukDurumu = 'bos' | 'secili' | 'dolu'` union tipinin Python karşılığıdır. `Literal`, bir alanın yalnızca belirtilen sabit değerlerden birini alabileceğini söyler; başka bir metin (ör. `"rezerve"`) gönderilirse `422` doğrulama hatası oluşur. Burada önemli bir nüans vardır: `'secili'` değeri Hf.4-5'te yalnızca tarayıcı tarafında (bir koltuğun henüz sunucuya gönderilmemiş seçim durumu) anlamlıydı; bu haftanın sunucusu ise yalnızca `'bos'` ↔ `'dolu'` arasında geçiş yapar. Tip tutarlılığı için `Literal` tanımında üç değer de kalır, ama sunucu mantığı — Ünite 3'te göreceğiniz gibi — yalnızca ikisini kullanır.

Ortak Hata / Tuzak

Pydantic'in v1 API'siyle (`class Config`, `.dict()`) v2 API'sini (`model_config`, `Field`, `model_dump()`) karıştırmayın — bu iki sürüm birbiriyle uyumsuzdur. Bu ders notunda ve `kod_ornekleri/hafta06/` klasöründeki tüm kod v2 sözdizimini kullanır (`pydantic~=2.13`); internette bulacağınız eski örneklerin bir kısmı hâlâ v1 sözdizimini gösterebilir.

6.2.4 422 doğrulama hatasının anatomisi

doğrulama hatası: Gelen verinin tanımlı tip/kısıtlara uymadığında dönen 422 cevabı (*validation error*).

Gelen verinin tanımlı tip/kısıtlara uymadığı her durumda FastAPI'nin otomatik ürettiği bu tür bir cevaba **doğrulama hatası** denir. `EtkinlikOlustur` gövdesine `"kapasite": -3` göndererek bu mekanizmayı gerçekten çalışırken görün:

Terminal

```
$ curl -i -X POST http://127.0.0.1:8000/etkinlikler \
    -H "Content-Type: application/json" \
    -d '{"baslik": "Konser", "tarih": "2026-11-14", "mekan": "Salon",
         "kapasite": -3, "biletFiyati": 250}'
```

Beklenen Çıktı

```
HTTP/1.1 422 Unprocessable Entity
content-type: application/json

{"detail":[{"type":"greater_than","loc":["body","kapasite"],
"msg":"Input should be greater than 0","input":-3,"ctx":{"gt":0}}]}
```

Cevap gövdesindeki `detail` bir dizidir (birden fazla alan aynı anda yanlış olabileceği için); her öğesi üç anahtar taşır: `loc` hatanın hangi alanda olduğunu gösterir (`["body", "kapasite"]` — gövdenin `kapasite` alanı); `msg` hatanın insan okunabilir açıklamasıdır ("Input should be greater than 0"); `type` ise hatanın makine okunabilir TÜRÜdür (`greater_than`). Bu mekanizma, Ünite 1'in yol parametresi tip hatasıyla (`int_parsing` türü) aynı altyapıdan — Pydantic'in doğrulama motorundan — gelir. En önemli nokta şudur: doğrulama başarısız olduğunda işleyici fonksiyonunuz hiç çağrılmaz — nesne asla oluşturulmaz, depoya asla yazılmaz; sanki istek hiç gelmemiş gibi davranılır.

İpucu

`model_dump()` bir Pydantic modelini düz bir Python `dict`'ine çevirir. FastAPI dönüş değerini otomatik serileştirdiği için rota fonksiyonu içinde genelde elle çağırmanız gerekmez; ama Ünite 3'ün depo katmanında (bir modelden yeni bir modele veri aktarırken) ve testlerde modelin içeriğini incelemek için sık kullanılan bir metottur.

3 • Katılım — Şimdi Siz Deneyin

`EtkinlikOlustur` modeliyle `POST /etkinlikler`'e `{"baslik": "Konser", "tarih": "2026-11-14", "mekan": "Salon", "kapasite": -3, "biletFiyati": 250}` gönderin; cevabın durum kodunu ve gövdesindeki `detail` listesini inceleyin.

Geribildirim. `422 Unprocessable Entity` döner; `detail[0].loc` `["body", "kapasite"]`'yi, `detail[0].msg` "greater than 0" ihlalini gösterir; nesne depoya hiç yazılmaz — doğrulama, işleyici fonksiyon çağrılmadan önce, FastAPI'nin Pydantic katmanında gerçekleşir.

Kendinizi Sınayın 6.2

`EtkinlikOlustur` modelinde `kapasite: int = Field(gt=0)` iken bir istemci `kapasite: 0` gönderirse ne olur?

5 • İzleme — Aktarım — Başka Nerede?

Bugün kurduğunuz `KamelModel` taban sınıfı ve giriş/çıkış modeli ayrımı deseni, Hf.8'de SQLAlchemy modelleri eklendiğinde de (ORM modeli ≠ Pydantic modeli ayrımı) aynı mantıkla sürecek; Hf.10'un kullanıcı kayıt formunda (şifre alanı girişte var, çıkışta asla dönülmez) aynı giriş/çıkış modeli ayrımı kullanılacak.

6.3 CRUD, Durum Kodları, Hata Cevapları

1 • Ön-öğretim — Neden Önemli?

Modelleriniz artık gelen veriyi doğruluyor, ama henüz hiçbir yerde saklamıyor — her istek "havada" kalıyor. Ayrıca gerçek bir bilet sisteminde iki kullanıcı aynı koltuğu aynı anda almaya çalışabilir: birinci kullanıcı koltuğu aldıktan sonra ikinci kullanıcının aynı koltuğu tekrar alabilmesi kabul edilemez bir hatadır. Bu ünite hem veriyi saklamanın hem de bu tür bir iş kuralını uygulamanın yolunu gösteriyor.

1 • Ön-öğretim — Ön Bilgi

Hf.1 Ünite 3'ün iş katmanı kavramını (isteği işleyen mantık katmanı) hatırlayın. YG I/II'nin anahtar-değer/sözlük veri yapılarını da hatırlayın — Python'ın `dict[int, X]`'i, bir `std::unordered_map`'e kavramsal olarak benzer. Son olarak Ünite 2'nin giriş/çıkış modellerini hatırlayın — bu ünitede CRUD fonksiyonlarının hangi modeli parametre, hangisini dönüş tipi olarak kullandığını göreceksiniz.

6.3.1 Bellekte depo: üç sözlük, üç sayaç

Bellekte depo: Verinin yalnızca çalışan sürecin belleğinde (kalıcı olmadan) tutulduğu basit depo (*in-memory store*).

Veriyi tutmanın en basit yolu, YG I/II'den zaten tanıdığınız bir anahtar-değer yapısıdır: bir **Bellekte depo**. Etkinlik/Koltuk/Bilet için ayrı birer `dict[int, ...]` kullanılır — anahtar her kaydın `id`'si, değer ise modelin kendisidir. Her varlık türü kendi sayacını (`sonraki_id`) tutar; her yeni kayıt eklendiğinde bu sayaç bir artar, böylece her kayda benzersiz bir kimlik atanmış olur:

```python
class Depo:

   def __init__(self) -> None:
      self._etkinlikler: dict[int, Etkinlik] = {}
      self._koltuklar: dict[int, Koltuk] = {}
      self._biletler: dict[int, Bilet] = {}
      self._sonraki_etkinlik_id = 1
      self._sonraki_koltuk_id = 1
      self._sonraki_bilet_id = 1
```

`Depo` sınıfının tamamı bu üç sözlük/sayaç çiftinin üzerine Etkinlik/Koltuk/Bilet için ayrı ayrı CRUD metotları ekler (tam sınıf `kod_ornekleri/hafta06/sunucu/app/depo.py` dosyasındadır). Bu metotların ilk örneği `etkinlik_ekle`'dir:

```python
def etkinlik_ekle(self, veri: EtkinlikOlustur) -> Etkinlik:
   etkinlik = Etkinlik(id=self._sonraki_etkinlik_id, **veri.model_dump())
   self._etkinlikler[etkinlik.id] = etkinlik
   self._sonraki_etkinlik_id += 1
   return etkinlik
```

Buradaki `Etkinlik(id=..., **veri.model_dump())` satırı, Ünite 2'nin `model_dump()` ipucunu somutlaştırır — `EtkinlikOlustur` modelinin alanlarını düz bir `dict`'e çevirip yeni bir `Etkinlik` nesnesine (bu kez `id` dahil) aktarır. `koltuk_guncelle` metodu ise (aynı dosyada) `model_copy(update={...})` kullanır — var olan bir nesneyi tamamen yeniden oluşturmak yerine, yalnızca `durum` alanını değiştirilmiş bir kopyasını üretir (Hf.4'ün React'teki değişmezlik ilkesine benzer bir desen: nesneyi yerinde değiştirmek yerine yeni bir nesne oluşturmak). Aynı

desen `bilet_iptal_et` metodunda da görünür — ama burada bir fark vardır: bir bileti depodan hiç silmez, yalnızca `durum`'unu `'iptal'`e çevirir ve ilgili koltuğu `'bos'`a döndürür:

```
def bilet_iptal_et(self, bilet_id: int) -> Bilet | None:
    bilet = self._biletler.get(bilet_id)
    if bilet is None:
        return None
    guncel_bilet = bilet.model_copy(update={"durum": "iptal"})
    self._biletler[bilet_id] = guncel_bilet
    koltuk = self._koltuklar.get(bilet.koltuk_id)
    if koltuk is not None:
        self._koltuklar[koltuk.id] = koltuk.model_copy(update={"durum": "bos"})
    return guncel_bilet
```

Kayıt silme yerine `durum` değiştirme, gerçek bir bilet sisteminde iptal edilen kayıtların denetim (*audit*) amacıyla saklanması gerektiği fikrinin bir yansımasıdır.

Analoji

Bellekte depoyu bir toplantı odasındaki beyaz tahtaya benzetin: üzerine yazdığınız her şey oda açıkken görünür ve paylaşılır (herkes aynı tahtayı okur), ama oda temizlendiğinde (sunucu yeniden başladığında) tahtadaki her şey silinir — kalıcı bir defter (veritabanı) değildir.

Tüm rota fonksiyonlarının aynı depoyu paylaşması için, modül düzeyinde tek bir örnek oluşturulur:

```
depo = Depo()
```

Ortak Hata / Tuzak

Sunucuyu `--reload` bayrağıyla çalıştırırken kaynak dosyalarınızda bir değişiklik yaptığınızda `uvicorn` süreci yeniden başlar — bu, `depo = Depo()` satırının tekrar çalışması, yani deponun baştan, boş olarak yeniden oluşturulması anlamına gelir. Az önce `curl` ile eklediğiniz bir etkinlik, dosyayı kaydettiğiniz anda sessizce kaybolur — bu bir hata değildir, bu haftanın bilinçli sınırı olan "kalıcılık yok" gerçeğinin doğal bir sonucudur (kalıcılık Hf.8'in konusudur).

6.3.2 Durum kodları: `POST`, `PUT`, `PATCH`, `DELETE`

Yönlendirici (router): Belirli bir varlığa/özelliğe ait rotaları grupladığı ayrı bir modül (*router*).

Rotalarınızı tek bir dosyada toplamak yerine, her varlık için ayrı bir **Yönlendirici (router)** tanımlarsınız — YG II'de büyük bir `.cpp` dosyasını sorumluluğa göre birden fazla dosyaya bölme disiplininin FastAPI karşılığıdır (dosya = sorumluluk sınırı):

```
router = APIRouter(prefix="/etkinlikler", tags=["etkinlikler"])
```

`prefix="/etkinlikler"` bu router'daki tüm yolların başına otomatik olarak `/etkinlikler` ekler; `tags=["etkinlikler"]` ise Ünite 4'te göreceğiniz `/docs` sayfasında bu rotaların hangi başlık altında gruplanacağını belirler. Yeni bir etkinlik oluşturma rotası şöyledir:

```
@router.post("", response_model=Etkinlik, status_code=201)
def etkinlik_olustur(
    veri: EtkinlikOlustur, depo: Depo = Depends(get_depo)
```

```
) -> Etkinlik:
    return depo.etkinlik_ekle(veri)
```

`status_code=201` dekoratöre verilen bir parametredir — bu satır olmasaydı FastAPI varsayılan olarak `200`'ü kullanırdı. `201 Created`, HTTP standardında özel olarak "yeni bir kayıt başarıyla oluşturuldu" anlamına gelir; cevap gövdesinde oluşturulan nesnenin tamamı (artık `id` dahil) döner. Bir kaydı okumak için — kayıt yoksa `404` döndürmek zorunda olduğunuz bir örnek — şu rotaya bakın:

```
@router.get("/{etkinlik_id}", response_model=Etkinlik)
def etkinlik_detay(etkinlik_id: int, depo: Depo = Depends(get_depo)) ->
    Etkinlik:
    etkinlik = depo.etkinlik_getir(etkinlik_id)
    if etkinlik is None:
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    return etkinlik
```

`HTTPException(status_code=404, detail=...)`, Python'ın `try`/`except` mekanizmasına benzer şekilde fırlatılan, ama doğrudan bir HTTP durum koduna bağlı bir istisnadır — FastAPI bu istisnayı yakalayıp otomatik olarak uygun bir JSON hata gövdesine (`{"detail": "..."}`) çevirir; siz bu dönüşümü elle yazmazsınız.

Bir kaydın tüm alanlarını yeni değerlerle değiştirmek için `PUT` kullanılır — bu hafta yalnızca `Etkinlik` için:

```
@router.put("/{etkinlik_id}", response_model=Etkinlik)
def etkinlik_guncelle(
    etkinlik_id: int, veri: EtkinlikOlustur, depo: Depo = Depends(get_depo)
) -> Etkinlik:
    etkinlik = depo.etkinlik_guncelle(etkinlik_id, veri)
    if etkinlik is None:
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    return etkinlik
```

`PUT`'un gövde tipi `EtkinlikOlustur` ile aynı şekli taşır — çünkü `PUT` bir kaydın tüm alanlarını (kimliği hariç) yeni baştan belirtmenizi bekler; eksik bıraktığınız bir alan varsa (ör. tarihi göndermezseniz) `422` doğrulama hatası alırsınız, eski değer korunmaz. Buna karşılık `Koltuk`'un yalnızca `durum` alanını değiştirmek için `PATCH` kullanılır:

```
@router.patch("/koltuklar/{koltuk_id}", response_model=Koltuk)
def koltuk_durum_guncelle(
    koltuk_id: int, veri: KoltukGuncelle, depo: Depo = Depends(get_depo)
) -> Koltuk:
    koltuk = depo.koltuk_guncelle(koltuk_id, veri.durum)
    if koltuk is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    return koltuk
```

`PATCH`'in gövde tipi (`KoltukGuncelle`) tek bir alan (`durum`) taşır — `PUT` ile `PATCH` arasındaki fark tam olarak budur: "tümünü değiştir" (`PUT`) ile "yalnızca belirtileni değiştir" (`PATCH`). Bu iki metot bilinçli olarak iki farklı varlıkla, en doğal kullanım örnekleriyle gösterilir — bir etkinliğin başlığını değiştirmek tam bir güncellemedir, bir koltuğun durumunu değiştirmek ise kısmi bir güncellemedir.

Son olarak silme işlemleri `204 No Content` ile cevap verir — ama iki farklı davranış sergiler. `Etkinlik` ve `Koltuk` için silme kaydı depodan tamamen kaldırır:

```
@router.delete("/{etkinlik_id}", status_code=204)
def etkinlik_sil(etkinlik_id: int, depo: Depo = Depends(get_depo)) -> None:
    silindi = depo.etkinlik_sil(etkinlik_id)
    if not silindi:
```

```
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
```

`Bilet` için ise "silme" aslında bir iptaldir — kaydı depodan silmez, yalnızca durumunu değiştirir ve ilgili koltuğu boşaltır:

```
@router.delete("/{bilet_id}", status_code=204)
def bilet_iptal(bilet_id: int, depo: Depo = Depends(get_depo)) -> None:
    bilet = depo.bilet_iptal_et(bilet_id)
    if bilet is None:
        raise HTTPException(status_code=404, detail="Bilet bulunamadı")
```

Her iki durumda da cevap gövdesiz `204`tür — silinen/iptal edilen bir şeyin içeriğini geri döndürmek anlamsızdır. Şekil 6.3 Etkinlik/Koltuk/Bilet için tüm metot×rota×durum kodu eşlemesini tek bir referans şemada toplar.

> **Dikkat**
>
> Bir `Etkinlik` silindiğinde ona bağlı `Koltuk`/`Bilet` kayıtları depoda değişmeden kalır — yani `etkinlik_id`'si artık var olmayan bir etkinliğe işaret eden koltuklar/biletler ortada kalabilir. Bu bilinçli bir sadeleştirmedir (*cascade delete* bu haftanın kapsamı dışındadır); ilişkisel silme/veri bütünlüğü kısıtları Hf.8'in SQLAlchemy ilişkiler ünitesinin konusudur.

6.3.3 Koltuk rezervasyonu iş kuralı: `409 Conflict`

Bu ünitenin en önemli parçası, `POST /biletler` çağrıldığında çalışan iş kuralıdır — Hf.1 Ünite 3'ün "iş katmanı" kavramının somutlaşması:

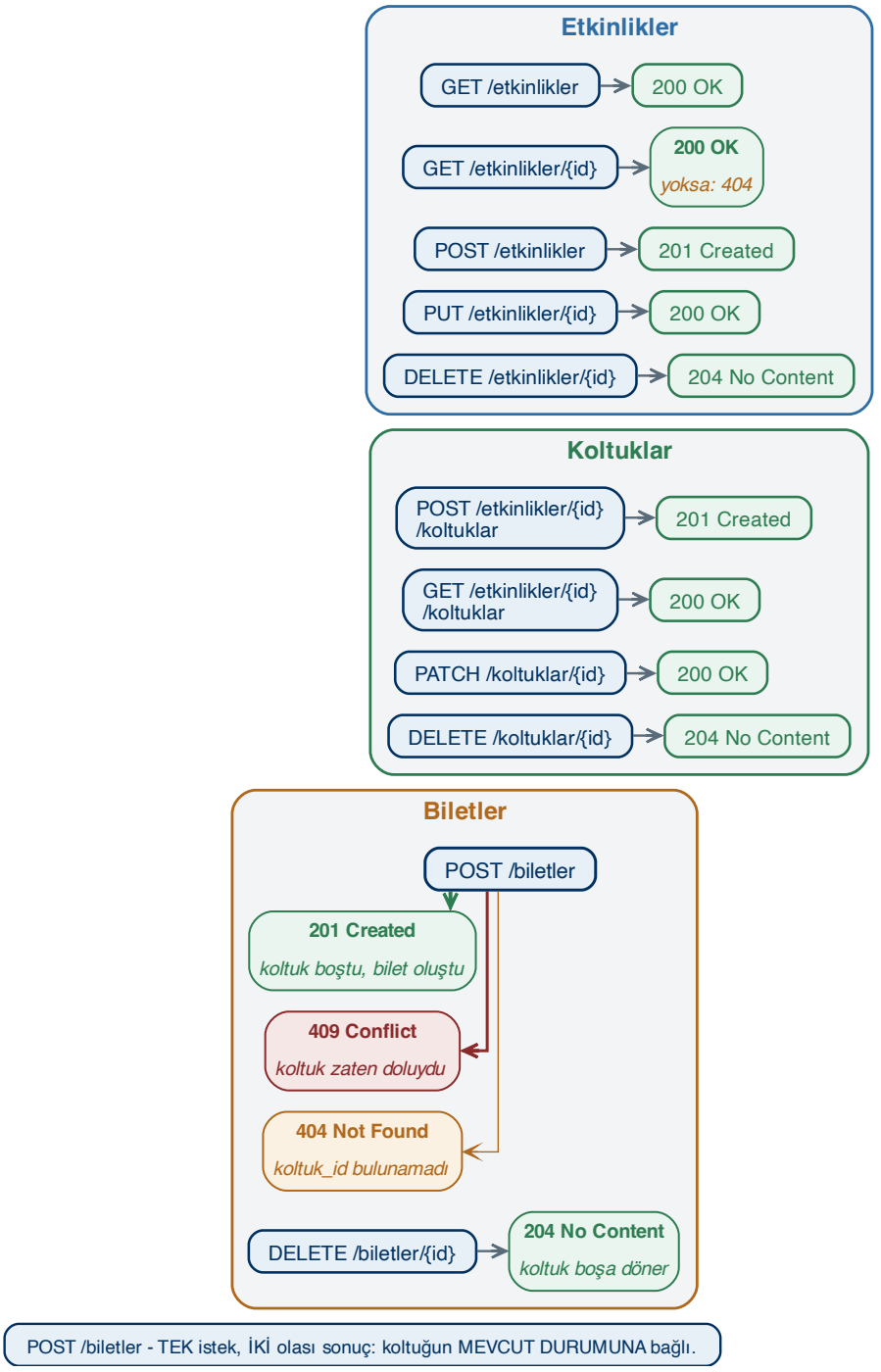
```
@router.post("", response_model=Bilet, status_code=201)
def bilet_olustur(veri: BiletOlustur, depo: Depo = Depends(get_depo)) ->
    Bilet:
    koltuk = depo.koltuk_getir(veri.koltuk_id)
    if koltuk is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    if koltuk.durum == "dolu":
        raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")
    bilet = depo.bilet_ekle(veri)
    depo.koltuk_guncelle(koltuk.id, "dolu")
    return bilet
```

Bu fonksiyon sırayla üç kontrol yapar. Önce ilgili `koltuk_id`'nin depoda var olup olmadığı kontrol edilir — yoksa `404`. Varsa koltuğun `durum`'u denetlenir: `'dolu'` ise istek **Çakışma (conflict) hatası** ile, yani `409 Conflict` durum koduyla reddedilir — "Bu koltuk zaten satılmış" gibi açıklayıcı bir `detail` mesajıyla. Koltuk `'bos'` ise bilet oluşturulur (`satin_alma_zamani` sunucu tarafında otomatik atanır), koltuğun `durum`'u `'dolu'`ya çevrilir ve `201` dönülür.

Çakışma (conflict) hatası: İsteğin biçimi geçerli ama sunucudaki mevcut durumla çeliştiği için reddedildiğini bildiren 409 durumu (*conflict (409)*).

> **Analoji**
>
> `409 Conflict`'i (koltuk çakışması) aynı numaralı otobüs koltuğunu iki kişinin ayrı gişelerden almaya çalışmasına benzetin: ikinci kişinin bileti biçim olarak tamamen doğru doldurulmuş olabilir, ama gişe memuru "bu koltuk zaten satıldı" diyerek ikinci satışı reddeder — sorun formatta değil, mevcut durumdadır.

Bu iş kuralını gerçek isteklerle doğrulayın — önce bir koltuk oluşturun, sonra aynı koltuğa iki kez bilet almayı deneyin:



Şekil 6.3: Etkinlik/Koltuk/Bilet için HTTP metodu, rota ve beklenen durum kodu eşlemesi; `POST /biletler`'in iki dallı sonucu (201/409) vurgulanmıştır.

```
curl -i -X POST "$BASE_URL/etkinlikler/1/koltuklar" \
  -H "Content-Type: application/json" \
  -d '{"sira": 1, "koltukNo": 5}'
```

Beklenen Çıktı

```
HTTP/1.1 201 Created
{"sira":1,"koltukNo":5,"id":1,"etkinlikId":1,"durum":"bos"}
```

```
curl -i -X POST "$BASE_URL/biletler" \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Ayşe Yılmaz"}'
```

Beklenen Çıktı

```
HTTP/1.1 201 Created
{"koltukId":1,"kullaniciAdi":"Ayşe Yılmaz","id":1,
 "satinAlmaZamani":"2026-09-07T14:42:30.690000","durum":"olusturuldu"}
```

```
curl -i -X POST "$BASE_URL/biletler" \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Mehmet Demir"}'
```

Beklenen Çıktı

```
HTTP/1.1 409 Conflict
{"detail":"Bu koltuk zaten satılmış"}
```

`400 Bad Request` ile `409 Conflict` arasındaki fark burada net görünür: ikinci istek format olarak tamamen geçerlidir — `koltuk_id` ve `kullanici_adi` doğru tiptedir, `422` hiç tetiklenmez — ama sunucudaki mevcut durumla (koltuk zaten satılmış) çelişir. `400` genel/format hatalarını ifade ederken, `409` özel olarak "isteğin mevcut kaynak durumuyla çeliştiği" anlamına gelir.

Son olarak üç router'ı tek bir uygulamada birleştirme adımına bakın:

```
app.include_router(etkinlikler.router)
app.include_router(koltuklar.router)
app.include_router(biletler.router)
```

Bu satırlardaki `include_router` çağrısı her router'ı ana uygulamaya bağlar. `koltuklar.router`'ın bir `prefix` taşımadığını fark edin — koltuk rotalarının bir kısmı bir etkinliğin altında (`/etkinlikler/.../koltuklar`), bir kısmı ise kendi başına (`/koltuklar/...`) yaşar; her router kendi yol yapısını kendisi belirler, `main.py` yalnızca onları birleştirir.

3 • Katılım **Şimdi Siz Deneyin**

Bir etkinlik ve bir koltuk oluşturun; `POST /biletler` ile bu koltuk için bir bilet alın (`201` bekleyin); aynı `koltuk_id` ile ikinci kez `POST /biletler` gönderin, durum kodunu ve `detail` mesajını gözlemleyin.

Geribildirim. İlk istek `201 Created` ile döner, koltuğun `durum`'u `'dolu'` olur; ikinci istek `409 Conflict` ile "Bu koltuk zaten satılmış" mesajı döner — koltuk zaten dolu olduğu için işleyici fonksiyon yeni bir bilet oluşturmadan erken çıkar.

Kendinizi Sınayın 6.3

Zaten `'dolu'` durumundaki bir koltuk için ikinci kez `POST /biletler` gönderirseniz ne olur; bu neden `400 Bad Request` değil `409 Conflict` ile cevaplanır?

5 • İzleme — Aktarım — Başka Nerede?

Bugünkü `HTTPException`/durum kodu disiplini, dönemin geri kalanında her hata durumunda (Hf.10'un "yanlış şifre" `401`'i, Hf.11'in yetkisiz erişim `403`'ü) aynı mekanizmayla ifade edilecek; koltuk rezervasyonu iş kuralı Hf.8'de "aynı anda iki isteğin yarışması" (*race condition*) sorunuyla yeniden karşınıza çıkacak — bu hafta bellekte tek bir Python sürecinde çalıştığınız için bu yarış durumu görünmez kalıyor, Hf.8 transaction/kilit mekanizmalarıyla bunu ele alacak.

6.4 OpenAPI Belgeleri ve Bağımlılık Enjeksiyonu Girişi

1 • Ön-öğretim — Neden Önemli?

API'niz çalışıyor, ama Hf.7'de bu API'yi tüketecek olan (siz ya da başka bir geliştirici) her uç noktanın tam olarak hangi alanları beklediğini/ döndürdüğünü nasıl bilecek? Elle bir belge yazmak, kod değiştikçe bu belgeyi güncel tutmayı gerektirir ve neredeyse her zaman geride kalır — bu ünite bu sorunu ortadan kaldıran bir mekanizma gösteriyor.

1 • Ön-öğretim — Ön Bilgi

Hf.3 Ünite 3'ün TS arayüzlerini hatırlayın — bu ünitede bunları sunucunun otomatik ürettiği bir belgeyle karşılaştıracaksınız. Ünite 1-3'te yazdığınız tüm rotalar/modeller de bu ünitenin ön bilgisidir — bu haftanın belgesi onlardan türetilir, elle yazılmaz.

6.4.1 `/docs` ve `/openapi.json`: kendiliğinden yazılan belge

Sunucunuz çalışırken tarayıcıda `http://127.0.0.1:8000/docs` adresini açtığınızda, yazdığınız her rotadan otomatik üretilmiş, interaktif bir belge sayfası (Swagger UI) görürsünüz — her uç noktayı genişletip "Try it out" ile gerçekten deneyebileceğiniz bir arayüz. Bu belge hiçbir ek kod yazmadan, yalnızca tip ipuçlarından ve Pydantic modellerinden türetilir.

Analoji

`/docs`'u bir restoranın, mutfak tarafından otomatik güncellenen menüsüne benzetin: siz yeni bir yemek (rota) eklediğinizde menü kendiliğinden güncellenir, garsonun (geliştiricinin) elle bir kağıda yeni yemeği yazmasına gerek kalmaz.

OpenAPI şeması: Bir API'nin rotalarını, parametrelerini ve veri şekillerini makine-okunabilir biçimde tanımlayan standart (*OpenAPI schema*).

Aynı belgenin makine-okunabilir hâli `http://127.0.0.1:8000/openapi.json` adresinde durur — bir **OpenAPI şeması**dır; her rotanın metodu, yolu, parametreleri, olası durum kodları ve gövde/yanıt şemalarını (Pydantic modellerinden üretilmiş) içerir. Bu şemadaki `Bilet` tanımının `properties` listesine bakın:

Beklenen Çıktı

```
"Bilet": {
  "properties": {
    "koltukId":        {"type": "integer"},
    "kullaniciAdi":     {"type": "string", "minLength": 1},
    "id":               {"type": "integer"},
    "satinAlmaZamani":  {"type": "string"},
    "durum": {"type": "string", "enum": ["olusturuldu", "iptal"]}
  }
}
```

Alan adlarının (`koltukId`, `kullaniciAdi`, `satinAlmaZamani`) Hf.3'ün `interface Bilet`'iyle birebir (camelCase) örtüştüğünü fark edin:

```
readonly id: number;
readonly koltukId: number;
kullaniciAdi: string;
satinAlmaZamani: string;
durum: BiletDurumu;
```

İki dilde yazılan aynı veri şeklinin ortak bir sözleşmede (OpenAPI şeması) buluştuğu fikri, Şekil 6.2'de Ünite 2'de gördüğünüz şemayla aynı köprüyü — bu kez rotaların/şemanın bütünü üzerinden — gösterir.

6.4.2 `Depends` ile bağımlılık enjeksiyonu

Bağımlılık enjeksiyonu: Bir fonksiyonun ihtiyaç duyduğu nesneyi dışarıdan alması (*dependency injection*).

Bağımlılık enjeksiyonu, Ünite 3'ün tüm rota fonksiyonlarında fark etmiş olabileceğiniz `depo: Depo = Depends(get_depo)` imzasının arkasındaki desendir: rota fonksiyonu, ihtiyaç duyduğu bir nesneyi (bu haftada: depo) kendisi oluşturmak yerine FastAPI'den ister. Çerçeve bu isteği her çağrıda karşılar, fonksiyon ise nesnenin nereden geldiğini bilmek zorunda kalmaz. `get_depo` fonksiyonu şöyle tanımlanır:

```
def get_depo() -> Depo:

   return depo
```

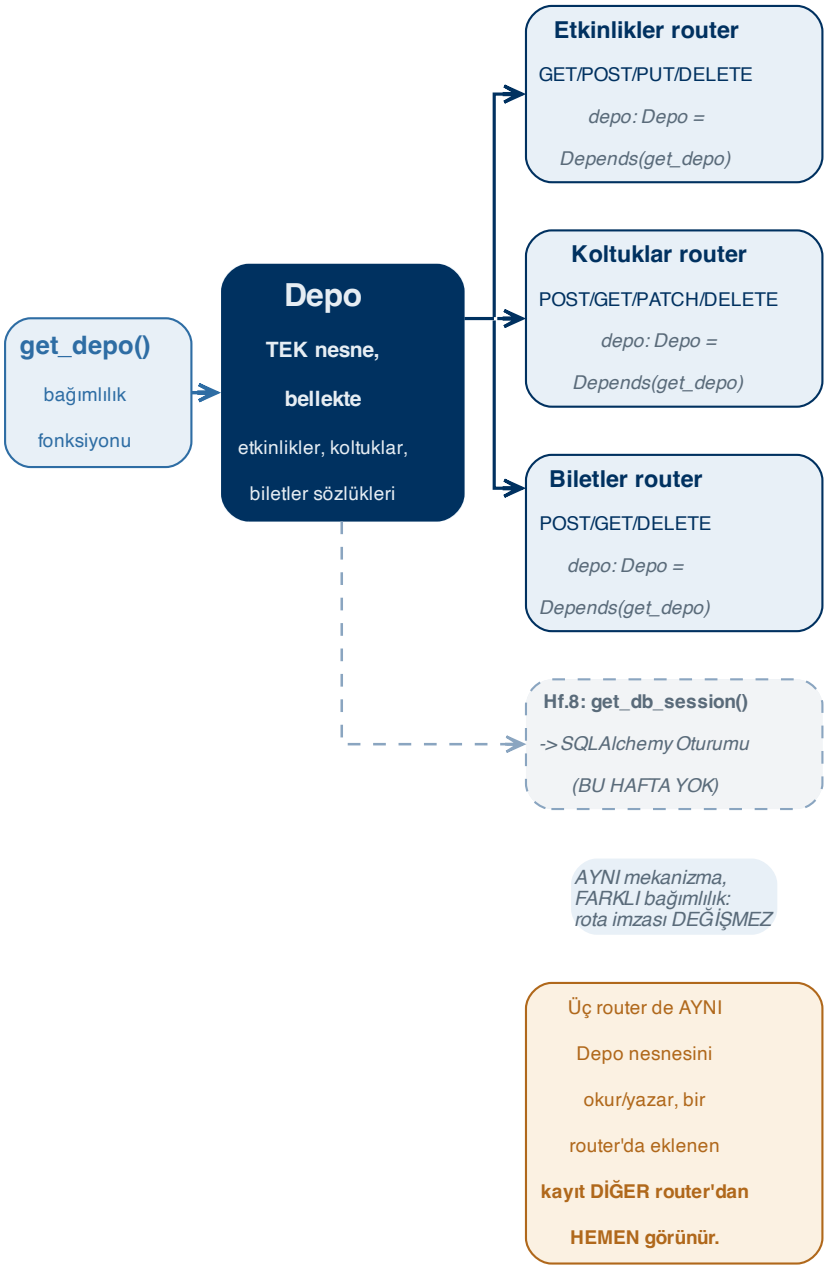
FastAPI her istekte `get_depo()` fonksiyonunu çağırıp sonucunu rota fonksiyonunun `depo` parametresine yerleştirir — rota fonksiyonu deponun nasıl oluşturulduğuyla hiç ilgilenmez, yalnızca "bana bir depo lazım" der. `get_depo()`'nun `app.depo` modülündeki tek, paylaşılan `depo` nesnesini döndürdüğünü hatırlayın (Ünite 3, `depo_ornegi`) — bu sayede tüm rotalar (Etkinlikler/Koltuklar/ Biletler) aynı depoyu kullanır, bir `POST /etkinlikler`'de eklenen etkinlik hemen ardından bir `GET /etkinlikler`'de görünür.

Analoji

`Depends`'i bir resepsiyon görevlisine, ihtiyaç duyduğu kayıt defterini kendisi satın almak yerine, binadan hazır verilmesine benzetin: görevli defterin nereden geldiğiyle ilgilenmez, yalnızca "bana bir defter lazım" der; bina yönetimi (FastAPI) hangi defterin (bellek deposu mu, ileride veritabanı oturumu mu) verileceğine karar verir.

Şekil 6.4, `get_depo()`'nun tek bir `Depo` nesnesini üç ayrı router'a nasıl enjekte ettiğini ve Hf.8'in `get_db_session()` yer tutucusunu gösterir.

`Depends(get_depo)` bu hafta bir bellek nesnesi enjekte ediyor; Hf.8'de SQLAlchemy'ye geçildiğinde aynı mekanizma, bu kez bir veritabanı oturumu



Şekil 6.4: `get_depo()` bağımlılık fonksiyonunun tek bir `Depo` nesnesini üç router'a enjekte etmesi ve Hf.8'in `get_db_session()` yer tutucusu.

(`Depends(get_db_session)`) enjekte etmek için kullanılacak — rota fonksiyonlarının bir bağımlılık parametresi alma biçimi değişmeyecek, yalnızca bağımlılığın ne olduğu değişecek.

> **İpucu**
>
> Sunucunuz çalışırken `/docs` üzerinden bir `POST /biletler` denediğinizde, tarayıcıdan gelen isteğe (Hf.7'nin konusu olan) bir engelle (CORS) karşılaşmazsınız — `/docs` sayfası aynı sunucudan sunulduğu için tarayıcının "farklı kaynak" kuralına takılmaz. Bu haftadan sonra tarayıcıdan (React) bu API'ye `fetch` ile bağlanmaya çalıştığınızda bir engelle karşılaşacaksınız — çözümü Hf.7'nin konusudur.

6.4.3 `Depends`'in testte değeri

Bağımlılık enjeksiyonunun asıl faydası test edilebilirlikte ortaya çıkar. Eğer rota fonksiyonları depoyu kendileri oluştursaydı — `Depends` olmasaydı — testler arasında veri sızıntısı (bir testin oluşturduğu kaydı bir sonraki testin görmesi) engellenemezdi. FastAPI `app.dependency_overrides` adlı bir sözlük sunar — bu sözlüğe `get_depo: lambda: Depo()` eklediğinizde, tüm testler boyunca `Depends(get_depo)` çağrıları gerçek `get_depo`'ya değil, bu yeni, boş depoya yönlendirilir:

```
@pytest.fixture
def client():
    test_depo = Depo()
    app.dependency_overrides[get_depo] = lambda: test_depo
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()
```

Her testte `test_depo = Depo()` satırıyla taze, boş bir depo oluşturulur; `app.dependency_overrides[get_depo] = lambda: test_depo` satırı bu depoyu `get_depo`'nun yerine koyar; `yield` sonrası `app.dependency_overrides.clear()` bu geçersiz kılmayı temizler, böylece bir sonraki test (bu `client` fixture'ını tekrar istediğinde) yeniden boş bir depoyla başlar.

6.4.4 İlk API testi: `TestClient`

Test istemcisi: Sunucu ayağa kaldırmadan uygulamaya doğrudan istek gönderen test aracı (*test client*).

Test istemcisi, gerçek bir sunucu ayağa kaldırmadan, doğrudan Python içinde HTTP benzeri istek göndermenizi sağlar; FastAPI'de bu araç `TestClient`'tır. YG II'nin `assert` disiplini burada aynen devam eder — yalnızca artık bir HTTP cevabının durum kodunu ve gövdesini denetlersiniz. Aşağıdaki test, bir etkinlik oluşturup listede göründüğünü doğrular:

```
def test_etkinlik_olustur_listele(client):
    yeni = {
        "baslik": "Konser",
        "tarih": "2026-11-14",
        "mekan": "KTÜ Kültür Merkezi",
        "kapasite": 100,
        "biletFiyati": 250,
    }
    cevap = client.post("/etkinlikler", json=yeni)
    assert cevap.status_code == 201
    olusturulan = cevap.json()
```

```
    assert olusturulan["id"] == 1
    assert olusturulan["biletFiyati"] == 250

    liste_cevabi = client.get("/etkinlikler")
    assert liste_cevabi.status_code == 200
    assert len(liste_cevabi.json()) == 1
```

Bu test tek bir fonksiyonda hem bir `POST` hem bir `GET` denetler.

`client.post("/etkinlikler", json=yeni)` çağrısı önce bir etkinlik oluşturur, cevabın `201` olduğunu ve gövdesindeki `id`/`biletFiyati` alanlarını doğrular; ardından `client.get("/etkinlikler")` ile listeyi çeker, listenin tam olarak bir öğe içerdiğini denetler. `client` parametresinin — test fonksiyonunun bir argümanı gibi görünen — aslında bir önceki kesitte gördüğünüz `conftest.py` fixture'ından geldiğini, `pytest`'in bu bağlantıyı fonksiyon adından otomatik kurduğunu unutmayın.

Tüm test paketini terminalde şöyle çalıştırırsınız:

Terminal

```
$ python -m pytest -q
```

Beklenen Çıktı

```
.........                                                [100%]
9 passed in 0.04s
```

Karmaşıklık (big-O)

`TestClient` ile yazılan testler gerçek bir TCP/HTTP bağlantısı açmaz — doğrudan Python içindeki FastAPI uygulama nesnesini çağırır. Bu yüzden bir test paketi, gerçek bir `uvicorn` süreci başlatıp gerçek ağ istekleri gönderen bir alternatife göre çok daha hızlı çalışır (milisaniyeler); bu hız, dönem boyunca sık sık `python -m pytest` çalıştırmanızı — her küçük değişiklikten sonra bile — pratik kılar.

3 • Katılım **Şimdi Siz Deneyin**

Sunucuyu çalıştırıp tarayıcıda `http://127.0.0.1:8000/docs` adresini açın; `POST /biletler` uç noktasını genişletip "Try it out" ile örnek bir istek gönderin; ardından `http://127.0.0.1:8000/openapi.json`'daki `Bilet` şemasının alan adlarını Hf.3'ün `interface Bilet`'iyle karşılaştırın.

Geribildirim. `/docs` sayfasında hiçbir ek kod yazmadan tüm rotalarınızı, beklenen gövde şemalarını ve olası durum kodlarını (201/404/409) görürsünüz; `openapi.json`'daki `Bilet` şemasının `properties` listesi (`id`, `koltukId`, `kullaniciAdi`, `satinAlmaZamani`, `durum`) Hf.3'ün `interface Bilet` alanlarıyla birebir (camelCase) örtüşür.

Kendinizi Sınayın 6.4

`Depends(get_depo)` yerine her rota fonksiyonunun içinde doğrudan `Depo()` çağırıp yeni bir depo nesnesi oluştursaydınız ne olurdu?

5 • İzleme **Aktarım — Başka Nerede?**

`Depends` ile bağımlılık enjekte etme deseni Hf.8'de `Depends(get_db_session)`'a, Hf.10'da `Depends(get_mevcut_kullanici)` (giriş yapmış kullanıcıyı enjekte eden bir bağımlılık) olarak genişleyecek; bugün yazdığınız ilk testler Hf.8'in transaction testlerinde, Hf.14'ün CI adımında (`pytest`) aynen çalışmaya devam edecek. Etkinlik/Bilet Sisteminin sunucusu artık paylaşılan, kendi kendini belgeleyen ve testlerle doğrulanmış bir temele kavuştu — Hf.7'de bu temele React ön yüzünüz `fetch` ile bağlanacak.

Bölüm Özeti

Özet

Bu hafta Hf.5'in "`Bilet[]` yalnızca tarayıcı sekmenizin belleğinde yaşıyor" sınırını ortadan kaldıran ilk gerçek sunucunuzu yazdınız. İlk ünitede `venv`+`pip`+`uvicorn` iş akışını kurup ilk `@app.get` rotanızı çalıştırdınız; yol parametresinin (`{}` içindeki değişken kısım) sorgu parametresinden (isteğe bağlı `?ad=deger`) farkını ve FastAPI'nin bir yol parametresini otomatik olarak nasıl dönüştürdüğünü/doğruladığını gördünüz. İkinci ünitede `BaseModel` tabanlı Pydantic modellerinin — Hf.3'ün TS arayüzlerinin aksine — çalışma zamanında veri doğruladığını, giriş (`EtkinlikOlustur`) ile çıkış (`Etkinlik`) modellerinin neden farklı şekiller taşıyabileceğini ve `KamelModel` taban sınıfının snake_case Python alanlarıyla camelCase JSON arasında nasıl köprü kurduğunu öğrendiniz. Üçüncü ünitede bir `Depo` sınıfıyla bellekte tam bir CRUD API'si yazdınız — `POST`→201, `PUT`→200, `PATCH`→200, `DELETE`→204 — ve koltuk rezervasyonu iş kuralını (`409 Conflict`) `HTTPException` ile uyguladınız. Dördüncü ünitede hiçbir ek kod yazmadan otomatik üretilen `/docs` belgesini, `Depends` ile bir depo nesnesinin rota fonksiyonlarına nasıl enjekte edildiğini ve `TestClient` ile ilk API testlerinizi gördünüz. Sonuçta Etkinlik/Bilet Sisteminin paylaşılan, testlerle doğrulanmış ilk sunucu katmanı ortaya çıktı. Gelecek hafta bu API'nin verisi hâlâ sunucunuzun belleğinde yaşarken, React ön yüzünüz `fetch` ile bu API'ye gerçekten bağlanacak.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — yol parametresi tip uyumsuzluğu). Yol şablonundaki `etkinlik_id: int` tip ipucuyla tanımlı bir rotaya `/etkinlikler/abc` isteği gönderdiğinizde `404 Not Found` değil, `422 Unprocessable Entity` döner. Nedeni şudur: `404`, yolun kendisi hiç tanımlı değilse (ör. hiç var olmayan bir `/boyle-bir-yol` isteseniz) dönerdi — burada ise yol şablonuyla birebir eşleşir, sorun yolda değildir. Sorun, FastAPI'nin URL'deki `"abc"` metnini `etkinlik_id` parametresinin tipi olan `int`'e çevirmeye çalışması ve bunu BAŞARAMAMASIdır — bu bir doğrulama hatasıdır, işleyici fonksiyonunuz hiç çağrılmadan önce, FastAPI'nin kendi katmanında oluşur.

Soru (Ünite 2 — `kapasite: 0` ile 422 anatomisi). `EtkinlikOlustur` modelinde `kapasite: int = Field(gt=0)` tanımlı iken bir istemci `"kapasite": 0` gönderirse `422 Unprocessable Entity` döner; `0`, `gt=0` ("0'dan büyük") kısıtını karşılamaz —

`0` sıfırdan büyük değildir. Cevap gövdesindeki `detail` dizisinin ilgili öğesinde `loc` alanı `["body", "kapasite"]` değerini taşır (hatanın gövdenin `kapasite` alanında olduğunu gösterir), `msg` alanı "Input should be greater than 0" benzeri bir açıklama, `type` alanı ise `greater_than` değerini taşır. Bu doğrulama, işleyici fonksiyon çağrılmadan önce gerçekleştiği için nesne depoya hiç yazılmaz — sanki istek hiç gelmemiş gibi davranılır.

Soru (Ünite 3 — ikinci rezervasyon neden 409, 400 değil). Zaten `'dolu'` durumundaki bir koltuk için ikinci kez `POST /biletler` gönderdiğinizde `409 Conflict` döner, "Bu koltuk zaten satılmış" benzeri bir `detail` mesajıyla. Bunun `400 Bad Request` değil `409 Conflict` olmasının nedeni şudur: istek format olarak tamamen geçerlidir — `koltuk_id` gerçekten bir tamsayıdır, `kullanici_adi` gerçekten bir metindir, `422` hiç tetiklenmez. Sorun isteğin biçiminde değil, sunucudaki mevcut durumla (koltuk zaten satılmış) çelişmesindedir. `400`, genel/format hatalarını ifade ederken; `409` özel olarak "isteğiniz geçerli ama kaynağın şu anki durumuyla çelişiyor" anlamına gelir — bu ayrım Hf.7'de React'in hangi hata türünü kullanıcıya nasıl göstereceğine karar vermesini sağlayacaktır.

Soru (Ünite 4 — `Depends` olmadan her rotada nesne oluşturmak). Bağımlılık enjeksiyonu yerine her rota fonksiyonunun içinde doğrudan `Depo()` çağırıp yeni bir nesne oluşturursaydınız, her istek kendi taze, boş deposunu oluştururdu — bir `POST /etkinlikler` isteğiyle eklediğiniz bir etkinlik, bir sonraki `GET /etkinlikler` çağrısında (o çağrının oluşturduğu depo bambaşka, boş bir nesne olduğundan) hiç görünmezdi — sanki hiçbir şey hiç kaydedilmemiş gibi davranırdı. `Depends(get_depo)` ise tüm rota fonksiyonlarına aynı, tek, paylaşılan `Depo` nesnesini enjekte ettiği için bu sorun hiç oluşmaz — bir router'da eklenen kayıt, diğer tüm router'lardan da hemen görünür.